

Avance del Proyecto Final — Pregunta 5

Arquitectura de Software

Subsistemas, librerías, interrupciones, diagramas de flujo, máquinas de estado, DMA

Datos del Estudiante

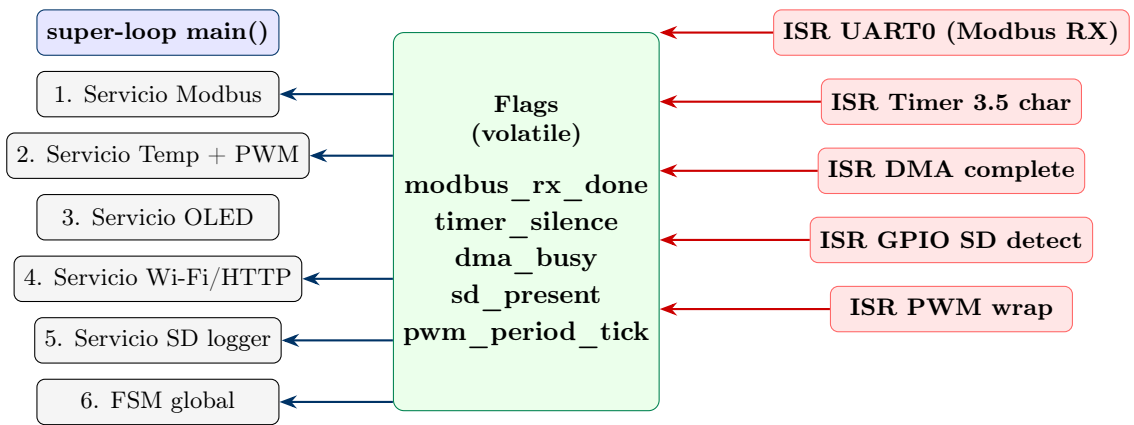
Estudiante: Brando Enrique Chávez Vergara (brandon.chavez@udea.edu.co)
Proyecto: Nodo de Edge Computing y Gateway IoT para Monitorización Industrial
Repositorio: https://github.com/kiikechavez/IoT_Edge_Gateway
SDK: Raspberry Pi Pico SDK – C/C++
Flujo: Polling + Interrupciones (único flujo válido)

1. Modelo de Programación: Polling + Interrupciones

Decisión de arquitectura

Acatando la directiva del profesor, el **único flujo de programa** implementado en este proyecto es **polling + interrupciones**. No se utilizan: RTOS multitarea, malloc dinámico, hilos, ni callbacks en background. El `main()` contiene un **super-loop** que sondea banderas (*flags*) modificadas por las rutinas de servicio a interrupción (ISR).

1.1. Esquema general



Las ISRs son cortas: actualizan banderas `volatile` y/o llenan buffers DMA. El super-loop sondea esas banderas y ejecuta el trabajo pesado fuera del contexto interrumpido.

2. Subsistemas y Librerías

2.1. Subsistemas internos del firmware

Subsistema	Módulo	Responsabilidad
temperature	temp_sensor.c	Lectura ADC4 + conversión a °C
pwm_ctrl	pwm_fan.c	Lazo proporcional $T \rightarrow \text{duty}$
modbus	modbus_rtu.c	Maestro Modbus RTU + DMA
oled	ssd1306.c	Driver pantalla OLED I2C
sdcard	sdcard_logger.c	Datalogger CSV (FatFs)
wifi	wifi_manager.c	Init y monitor del CYW43439
http	http_client.c	Cliente HTTP POST + JSON
statemachine	fsm_global.c	FSM INIT/ONLINE/OFFLINE/ERROR

2.2. Librerías externas (todas open-source)

- **Pico SDK** (oficial): hardware_adc, hardware_pwm, hardware_uart, hardware_i2c, hardware_spi, hardware_dma, hardware_timer, hardware_irq, pico_stdlib.
- **pico-extras** (CYW43): pico_cyw43_arch_lwip_poll.
- **lwIP** (incluido en SDK): pila TCP/IP en modo *NO_SYS*.
- **FatFs** (Chan): sistema de archivos FAT32 para microSD.
- **no-OS-FatFS-SD-SPI-RPi-Pico**: capa SPI de FatFs para Pico.
- **biblioteca estándar de C**: snprintf para JSON.

3. Mapa de Interrupciones

Fuente	Vector RP2040	Prioridad	Acción de la ISR
UART0 RX (Modbus)	UART0_IRQ	ALTA	Empuja byte recibido al buffer circular; reinicia timer 3.5 char.
Timer hardware 3.5 char	TIMER_IRQ_3	ALTA	Marca modbus_rx_done = true.
DMA canal Modbus TX	DMA_IRQ_0	MEDIA	Libera el bus y cambia DE/RE _n a recepción.
GPIO SD card detect	IO_IRQ_BANK0	BAJA	Marca sd_present = nueva lectura.
PWM wrap (canal 7)	PWM_IRQ_WRAP	BAJA	Incrementa contador para muestreo del lazo a 100 Hz.
CYW43 (Wi-Fi)	gestionado por SDK	N/A	cyw43_arch_poll() en el super-loop.

4. Uso de DMA

Justificación

El uso de DMA está directamente alineado con el requisito no funcional **RNF2 (precisión de tiempos Modbus)**, ya que libera al CPU de mover bytes durante la transmisión/recepción UART y permite sostener los tiempos de silencio inter-frame de 3.5 caracteres sin deriva.

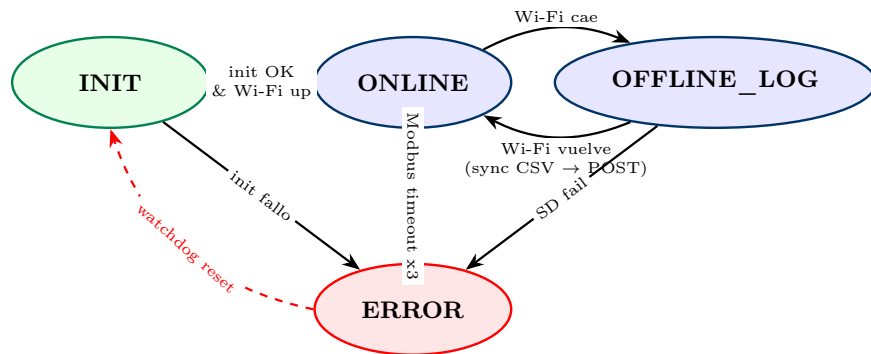
Se reservan **dos canales DMA**:

Canal	Dirección	Periferico	Uso
DMA0	MEM → UART0_TX	UART0	Envío automático de la trama Modbus de petición.
DMA1	UART0_RX → MEM	UART0	Recepción automática de la trama de respuesta a buffer circular.

Disparadores (*DREQ*) usados: `DREQ_UART0_TX` y `DREQ_UART0_RX` configurados con `dma_channel_set_trans` y `dma_channel_set_irq0_enabled()`.

5. Máquina de Estados Global del Sistema

Implementada en `src/statemachine/fsm_global.c`. Estados: `INIT`, `ONLINE`, `OFFLINE_LOG`, `ERROR`.

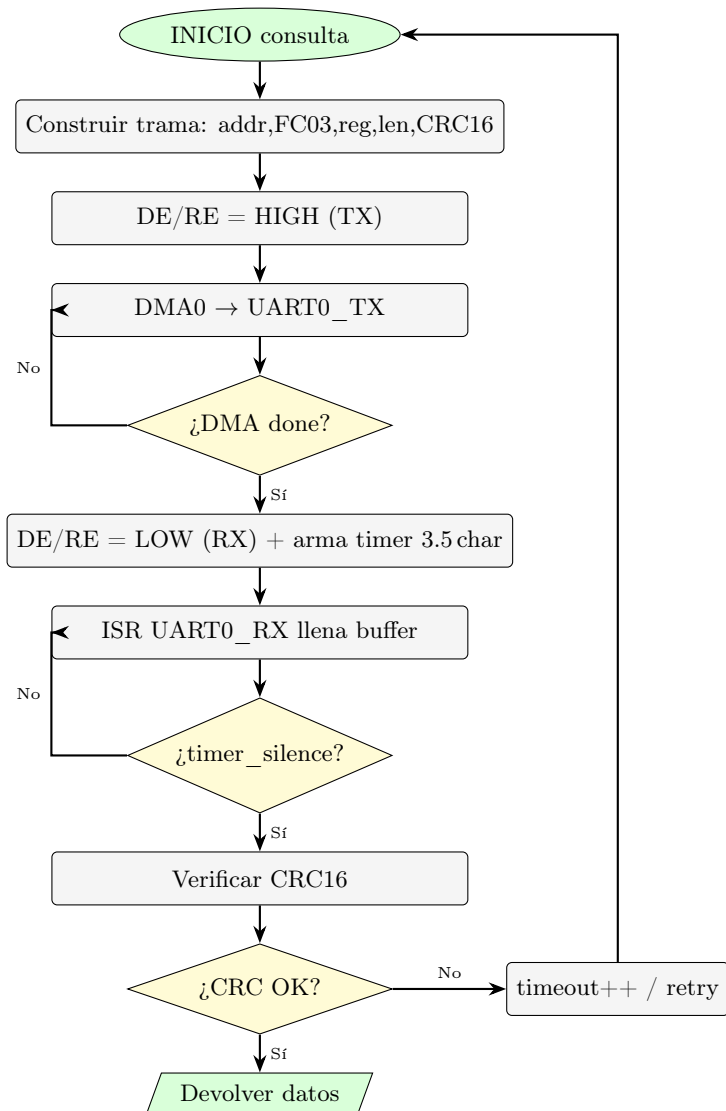


5.1. Tabla de transiciones

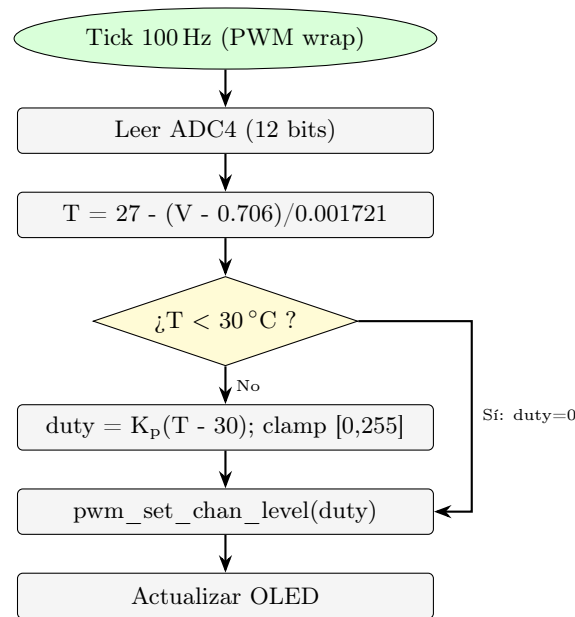
Estado actual	Evento (flag)	Acción / Estado destino
INIT	wifi_ok & periph_ok	→ ONLINE
INIT	error de inicialización	→ ERROR
ONLINE	wifi_link_lost	→ OFFLINE_LOG (abre archivo CSV)
ONLINE	modbus_timeout >= 3	→ ERROR
OFFLINE_LOG	wifi_reconnected	Sincroniza CSV; → ONLINE
OFFLINE_LOG	sd_fail	→ ERROR
ERROR	timer recovery (10s)	Watchdog reset → INIT

6. Diagramas de Flujo de los Algoritmos Principales

6.1. Algoritmo del maestro Modbus RTU (polling + ISR)



6.2. Algoritmo del lazo de control térmico (proporcional)



6.3. Algoritmo del super-loop main()

```

1  int main(void) {
2      stdio_init_all();
3      fsm_global_init();           // INIT
4      temp_sensor_init();         // ADC4
5      pwm_fan_init();             // PWM canal 7
6      oled_init();               // I2C
7      modbus_init();             // UART0 + DMA0/DMA1 + timer 3.5 char
8      sdcard_init();             // SPI1 + FatFs
9      wifi_manager_init();       // CYW43 + lwIP
10
11     while (true) {
12         // 1. Sondear flags actualizadas por ISRs
13         if (flag_modbus_rx_done) modbus_service();
14         if (flag_pwm_tick) temp_pwm_service();
15         if (flag_oled_refresh) oled_service();
16         if (flag_http_due) http_post_service();
17         if (flag_sd_write_pending) sdcard_service();
18
19         // 2. Mantener stack lwIP del CYW43
20         cyw43_arch_poll();
21
22         // 3. Avanzar la FSM global
23         fsm_global_step();
24
25         // 4. Dormir hasta el siguiente evento
26         __wfe(); // wait-for-event
27     }
28 }

```

7. Resumen de Cumplimiento

- ✓ Flujo polling + interrupciones aplicado en todos los módulos.
- ✓ 6 fuentes de interrupción identificadas y documentadas.
- ✓ 2 canales DMA usados para UART0 TX/RX (cumple RNF2).

- ✓ **FSM global** con 4 estados y tabla de transiciones documentada.
- ✓ **Diagramas de flujo** de los dos algoritmos críticos (Modbus + lazo PWM).
- ✓ **Arquitectura modular** en 8 subsistemas, cada uno con su propia rama **feature/***.